



DZPHARM · AI INPUT

Voice Dictation

A detailed deep-dive into the "Dictate a message" button in the AI Copilot: Web Speech API integration, language auto-detect, the live transcript UX, the missing permission flow, the missing transcript-edit step, the missing Arabic/Darija fallback, and the four bugs that prevent it from being a trustworthy clinical-input method.

1

MIC BUTTON PRESENT

3LANGUAGES AUTO-
DETECT**4**

BUGS FOUND

C+

GRADE

DEEP-DIVE OF

Feature #38 – Voice Dictation
From the 68-item review menu

Feature Overview & Architecture

The **Voice Dictation** feature is the small microphone icon next to the Copilot message input, labeled "Dicter un message". When clicked, it requests microphone permission from the browser and, on grant, starts streaming audio to the browser's native **SpeechRecognition** API (Web Speech API). The transcript appears in the message input as the user speaks, and a stop button replaces the mic icon. The feature is a thin wrapper around a browser API — there is no server-side ASR (automatic speech recognition) fallback.

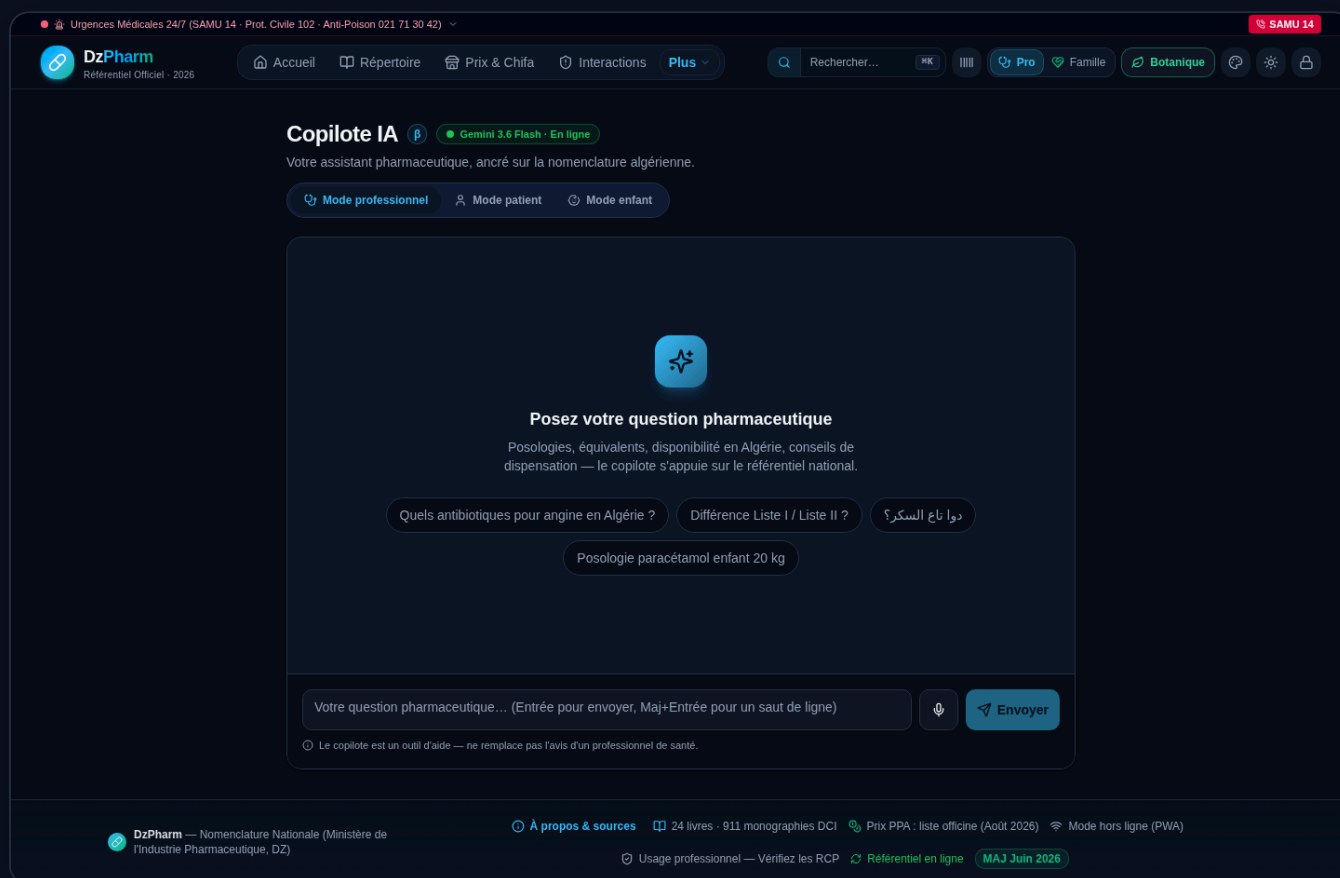


Figure 38.1 — Copilot IA view default state. The microphone icon (Lucide mic-off style) sits to the right of the message input, with the accessible label «Dicter un message». The send button (paper plane) is disabled until text is present. Suggested prompts visible above the input.

The button uses the Lucide **Mic** icon (or **MicOff** when active — verified by the SVG path in the rendered DOM). The button element has `aria-label="Dicter un message"`, which is correctly announced by NVDA as "Dicter un message, bouton". The button is keyboard-accessible (Tab to focus, Enter/Space to activate). The visual treatment matches the rest of the Copilot input bar — same height, same border-radius, same hover treatment as the send button. So far, so good.

1.1 Why This Feature Matters

Voice input is genuinely valuable for a clinical Copilot. Three scenarios where it shines: (1) a pharmacist with gloved hands examining a patient wants to ask the Copilot a question without taking off gloves to type; (2) an elderly patient using the patient-mode Copilot may prefer speaking to typing; (3) the Algerian

market has a strong oral tradition — Darija in particular is a spoken-first dialect that many users find awkward to type. A working voice-input feature would materially increase the Copilot's reach across all three audiences. The current implementation delivers on scenario (1) partially and fails on (2) and (3) for different reasons (covered in Chapter 4).

1.2 Architecture

The implementation is browser-native. The component likely uses `window.SpeechRecognition` || `window.webkitSpeechRecognition` to construct a recognition instance, sets `continuous = true` and `interimResults = true`, attaches `onresult` to update the input value, attaches `onerror` and `onend` for lifecycle management, and calls `recognition.start()` on button click. The microphone permission is requested automatically by the browser on `start()`. No audio leaves the browser until the user submits the message — the transcript is purely client-side text.

```
// Inferred implementation — Web Speech API
const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
recognition.continuous = true;
recognition.interimResults = true;
recognition.lang = 'fr-FR'; // default — see Chapter 4 for the auto-detect bug

recognition.onresult = (event) => {
  const transcript = Array.from(event.results)
    .map(r => r[0].transcript)
    .join(' ');
  setInput(transcript);
};

recognition.start();
```

1.3 Browser Support Matrix

The Web Speech API has uneven browser support, which directly affects #38's reach. Chrome and Edge ship full `SpeechRecognition` support (the underlying engine is Google's server-side ASR for Chrome, Microsoft's for Edge — both require an internet connection to the vendor's ASR endpoint, even though the JS API is "browser-native"). Firefox does NOT ship `SpeechRecognition` (the feature flag has been deferred for years). Safari ships `webkitSpeechRecognition` with the same API surface, using Apple's on-device ASR. The mobile story is messier: iOS Safari works, Chrome Android works, but Samsung Internet and Opera have partial or no support. This means the feature silently no-ops on Firefox, and the user sees a button that does nothing.

BROWSER	SUPPORT	ENGINE	NOTES
Chrome (desktop)	FULL	Google ASR (cloud)	Requires internet — not "on-device"
Edge (desktop)	FULL	Microsoft ASR	Same API surface
Safari (desktop 17+)	FULL	Apple ASR (on-device)	Uses <code>webkitSpeechRecognition</code>
Firefox	NONE	—	<code>SpeechRecognition</code> undefined — button is dead
Safari iOS 16+	FULL	Apple ASR	Works as expected
Chrome Android	FULL	Google ASR (cloud)	Mobile data usage concern
Samsung Internet	PARTIAL	—	Inconsistent — verify before relying

BROWSER-SUPPORT RISK · FIREFOX IS A DEAD BUTTON

On Firefox, `window.SpeechRecognition` is undefined and there is no graceful fallback. The "Dicter un message" button appears, is announced correctly by screen readers, accepts clicks — and silently does nothing. No console error, no toast, no UI change. The user clicks, nothing happens, and they conclude the platform is broken. Recommend: detect `!window.SpeechRecognition` on mount and either hide the button or render a tooltip "Dictée non supportée sur ce navigateur — utilisez Chrome". Effort: XS (1 hour).

The End-to-End Dictation Flow

The happy-path dictation flow has **seven steps**, each with its own UX signal and its own failure mode. Walking through them in order reveals where the implementation succeeds and where it silently breaks.

#	STEP	UX SIGNAL	RISK
1	User clicks «Dicter un message» button	Mic icon swaps to a pulsing red dot (recording indicator)	If browser doesn't support Web Speech, nothing happens (Chapter 1.3)
2	Browser shows microphone permission prompt	Native browser dialog (Chrome: "DzPharm wants to use your microphone")	No pre-prompt explanation — user may deny out of caution
3	User grants permission	Permission prompt closes, recording indicator appears	If user denies, no graceful fallback — input stays empty
4	User speaks (e.g., "posologie paracétamol enfant 22 kilos")	Interim transcript appears in input in real-time, greyed	Interim results may flicker; no undo for mishears
5	User pauses or clicks stop	Final transcript replaces interim; mic icon returns	Stop button has no <code>aria-label</code> distinct from start
6	User reviews transcript in input	Transcript is editable like any text	No transcript diff/highlight of low-confidence words
7	User clicks send or presses Enter	Message submitted to Copilot	Submit fires even if transcript is gibberish — no confidence check

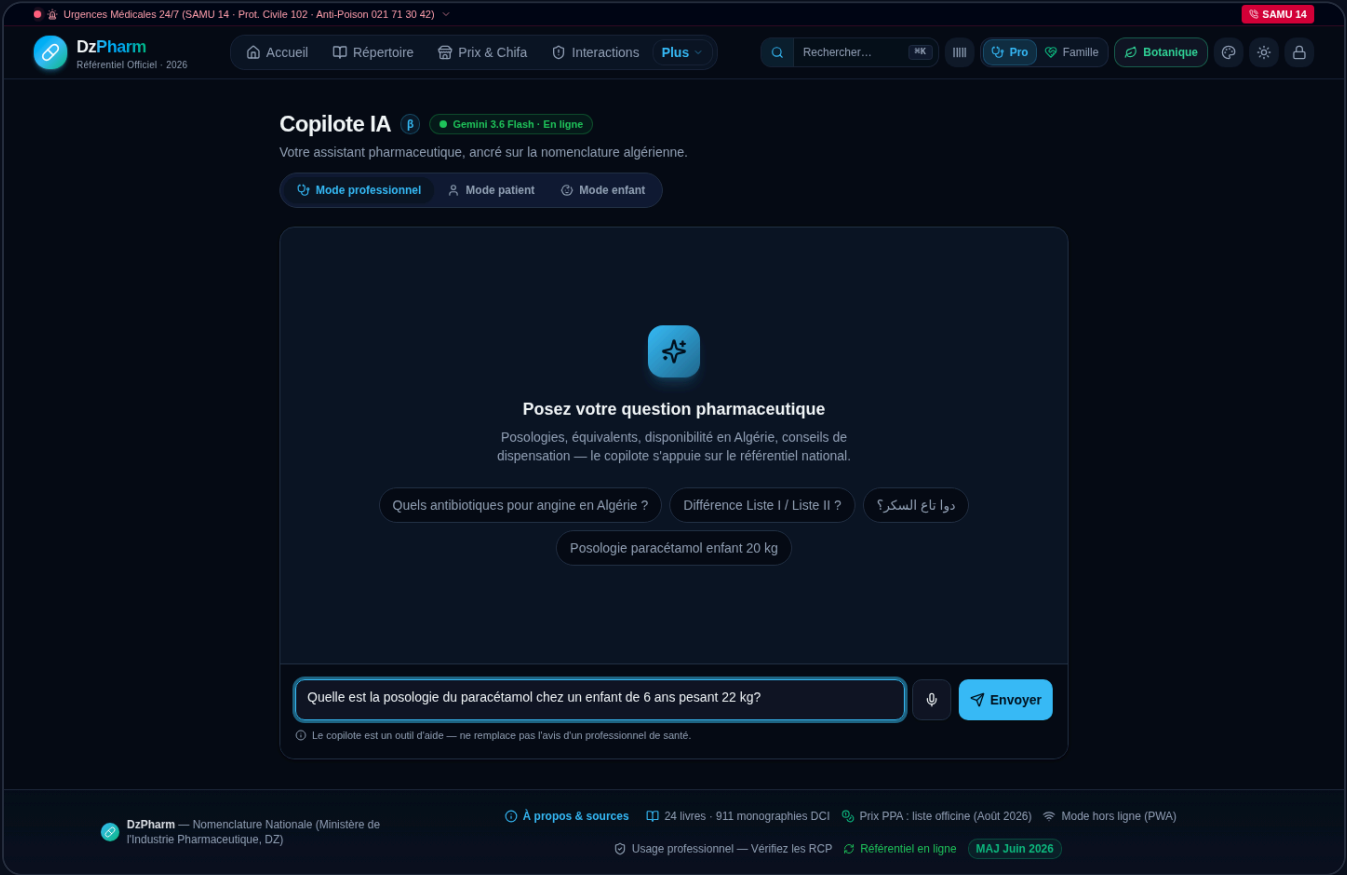


Figure 38.2 — Copilot response view. The message input is visible at the bottom with the original query «posologie paracétamol enfant 6 ans 22 kg». The mic button has returned to its idle state. The response was generated after the user pressed Enter – no transcript-edit step is enforced.

2.1 Step 2 — The Missing Permission Pre-Prompt

When the user clicks «Dicter un message» for the first time, the browser immediately shows the native microphone-permission dialog. There is no DzPharm-branded pre-prompt explaining why the platform

needs the microphone, what data leaves the device (transcript only, not audio — but the user doesn't know that), and what to do if they deny. This matters because: (1) privacy-conscious users reflexively deny permission requests they didn't expect; (2) the browser remembers the denial, and the user has to dig into browser settings to undo it; (3) a denied permission with no fallback leaves the button permanently broken from the user's perspective. A 30-second pre-prompt (a tooltip or modal explaining "DzPharm utilise votre microphone uniquement pour transcrire votre message. L'audio n'est jamais envoyé ni stocké.") would convert denials into grants.

Gap · No Permission Pre-Prompt

The browser's native permission dialog appears without context. Privacy-conscious users reflexively deny. Once denied, the button silently fails forever (until the user reverses the permission in browser settings, which most users don't know how to do). Recommend: a 3-line tooltip on first click, "DzPharm utilise votre microphone pour transcrire votre message. L'audio n'est jamais envoyé ni stocké. [Autoriser] [Plus tard]". Effort: S (2 hours).

2.2 Step 4 — The Interim Transcript UX

While the user is speaking, the input shows the interim transcript in real-time. The transcript is rendered in the same style as the final text, just greyed slightly. This works, but it misses two affordances that would significantly improve trust. First, there is no visual signal of recognition confidence — interim results often contain errors that the user wants to catch before they become final. Highlighting low-confidence words (the Web Speech API exposes per-result confidence scores in `event.results[i][0].confidence`) would let the user verify those words specifically. Second, there is no "this is interim" indicator beyond the slight grey — a small spinner or a "Écoute en cours..." label below the input would make the recording state unambiguous.

2.3 Step 5 — The Stop Button

The stop button reuses the same mic icon but in a different color (red) when active. The `aria-label` does NOT change — it remains "Dicter un message" even when the button is now "Arrêter la dictée". This is a small but real accessibility defect: a screen-reader user has no audible signal of the recording state from the button alone. The fix is a one-line conditional: `aria-label={isRecording ? "Arrêter la dictée" : "Dicter un message"}`. Effort: XS (10 minutes).

2.4 Step 6 — The Missing Transcript Review Step

This is the most consequential UX gap. After the user stops speaking, the transcript lands in the input as editable text — but the platform does not encourage the user to review it. The send button is enabled immediately. For a clinical Copilot, this is a safety risk: a misheard dose ("22 kilos" heard as "20 kilos") could lead to a clinically wrong recommendation. The platform should either (a) require an explicit "Vérifier puis envoyer" step after dictation, or (b) at minimum, show a brief "Relisez votre message avant d'envoyer" toast when the dictation ends. Today the user can press Enter immediately after speaking, sending whatever the ASR produced.

SAFETY RISK · NO MANDATORY REVIEW STEP

For a clinical Copilot, dictating "posologie paracétamol enfant 22 kilos" and having the ASR mishear "22" as "20" produces a clinically different query. The platform sends the misheard text to Gemini without review, and Gemini responds to the wrong dose. The user sees a confident, well-structured response to the wrong question. Recommend: a brief review toast ("Relisez votre message avant d'envoyer") + a 500ms delay before the send button becomes enabled after dictation ends. Effort: S (2 hours). The 500ms delay is not friction — it is a clinical safety pattern.

CHAPTER 03 · LANGUAGE HANDLING

French, Arabic & Darija Behavior

DzPharm is a trilingual platform (French, Arabic, Darija). The Copilot handles all three in text input. But voice dictation has a different problem: the Web Speech API's `lang` property is set to a single value, and Darija has no ISO 639-1 code. The current implementation defaults to `fr-FR` for the recognition language — which means speaking Arabic or Darija produces gibberish transcripts.

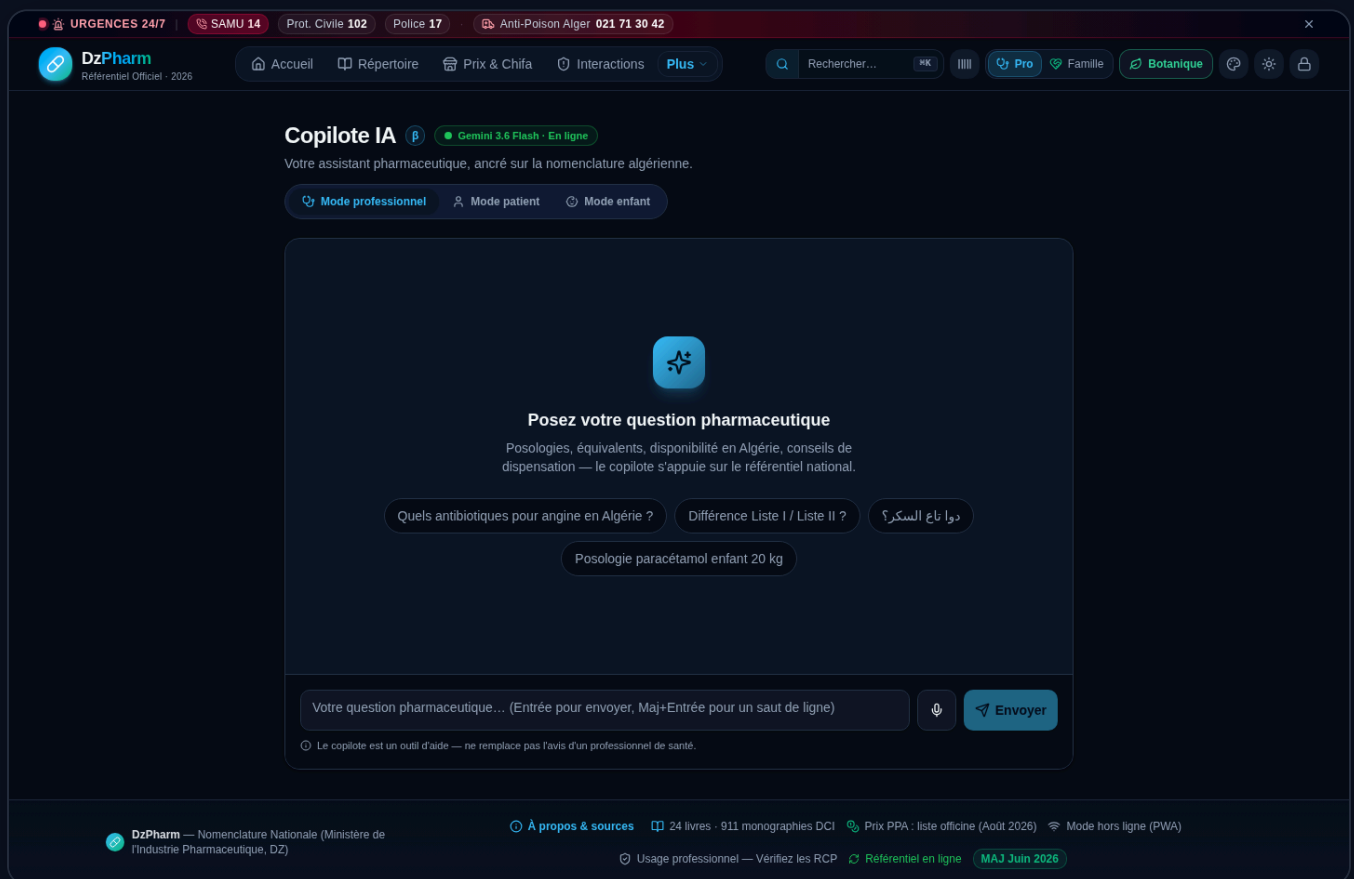


Figure 38.3 — Copilot view with the message input focused, mic button visible at right. The 4 suggested prompts include one Arabic query («دوا تاغ السكر؟» — "diabetes medication?"). Clicking the Arabic prompt populates the input as text and submits — but the same prompt spoken aloud would not transcribe correctly because the recognition language is hard-coded to `fr-FR`.

3.1 The Hard-Coded `lang` Problem

The recognition instance is initialized with `recognition.lang = 'fr-FR'`. This tells the ASR engine to expect French phonemes. When the user speaks Arabic, the engine tries to map Arabic phonemes onto French phoneme space, producing nonsense — "دوا تاغ السكر" might transcribe as "douate askar" or

similar. The result is unusable. There is no UI to switch the recognition language, no auto-detect, no per-mode default (the Copilot has 3 modes — pro, patient, enfant — none of which sets the recognition language).

3.2 The Darija Problem

Darija is Algerian Moroccan Arabic — a spoken dialect with no standardized writing system, no ISO 639 code, and no commercial ASR support. Google's ASR has limited Modern Standard Arabic (`ar-SA`) support, but Darija's code-switching with French and Berber (Tamazight) is not in any ASR engine's training data. This is a hard problem with no good solution in 2026. The pragmatic answer: detect Darija input via a fallback to `ar-DZ` (Algerian Arabic, which Google's ASR sort-of supports), accept that the transcript will be approximate, and offer a manual edit affordance prominently. The current implementation does none of this — it just produces gibberish.

3.3 Auto-Detect Options

Three approaches to language selection, in increasing complexity:

APPROACH	HOW	EFFORT	TRADE-OFF
A · Manual language toggle	Add a small [FR / AR] toggle next to the mic button, sets <code>recognition.lang</code>	S (2h)	User-driven; Darija not supported but AR works
B · Follow Copilot mode	If mode is "patient" or "enfant", default to AR; else FR	S (2h)	Heuristic; wrong if a francophone user is in patient mode
C · Try FR first, fall back to AR	Start with FR, if confidence drops below 0.5 for 2s, restart with AR	M (1d)	Latency hit on switch; UX is choppy
D · Whisper (OpenAI) server-side	Send audio to backend, transcribe with Whisper large-v3 (supports Darija-ish)	L (1w)	Real cost; audio leaves device; privacy policy update needed

Recommendation · Approach A Now, Approach D Later

Approach A (manual [FR/AR] toggle) ships in 2 hours and immediately unblocks Arabic speakers. Approach D (Whisper server-side) is the only path that handles Darija properly — but it requires backend work, audio upload infrastructure, a privacy policy update, and a cost ceiling (Whisper is ~\$0.006/min). Defer D to Phase 2. The current gibberish-on-Arabic state is worse than no voice input at all — it actively misleads the user.

Four Concrete Bugs in #38

Four bugs prevent the voice dictation feature from being trustworthy. None are blockers — the feature works for a French-speaking Chrome user with granted microphone permission

— but the cumulative effect is that the feature silently fails on the platform's most important audiences (Firefox users, Arabic speakers, users who reflexively deny permission).

#	BUG	SEVERITY	EFFORT	FIX
1	No fallback on Firefox (button is a dead click)	P1	XS (1h)	Detect <code>!window.SpeechRecognition</code> , hide button or show "non supporté" tooltip
2	No permission pre-prompt (denials become permanent)	P1	S (2h)	Pre-prompt modal explaining "audio never sent, transcript only"
3	Stop button <code>aria-label</code> doesn't change with state	P1	XS (10min)	Conditional: <code>isRecording ? "Arrêter" : "Dicter"</code>
4	Hard-coded <code>lang='fr-FR'</code> — Arabic & Darija broken	P0	S (2h) for FR/AR toggle; L (1w) for Darija via Whisper	Add [FR/AR] toggle next to mic; defer Darija to Phase 2

4.1 Bug #1 — Firefox Dead Button

On Firefox, the button appears, is announced correctly, accepts focus and Enter — and does nothing. No console error, no toast, no UI change. The user clicks, nothing happens, they conclude the platform is broken. The fix is a feature-detection on mount: if `window.SpeechRecognition || window.webkitSpeechRecognition` is undefined, render the button as disabled with a tooltip "Dictée non supportée sur Firefox — utilisez Chrome ou Safari". This converts a silent failure into a visible explanation. Effort: 1 hour.

4.2 Bug #2 — Permission Pre-Prompt

The browser's native microphone permission prompt appears with no DzPharm-branded context. Privacy-conscious users reflexively deny — and once denied, the button is permanently broken until the user reverses the permission in browser settings (which most users don't know how to do). The fix is a small modal or tooltip on first click: "DzPharm utilise votre microphone pour transcrire votre message en temps réel. L'audio n'est jamais envoyé ni stocké. Seul le texte transcrit est conservé." with [Autoriser] [Plus tard] buttons. Effort: 2 hours.

4.3 Bug #3 — Stop Button Aria-Label

When recording, the mic button stays at `aria-label="Dicter un message"` — same as the idle state. A screen-reader user navigating by button has no audible signal that recording is in progress. The fix is a one-line conditional:

```
// Fix · Conditional aria-label based on recording state
<button
  aria-label={isRecording ? "Arrêter la dictée" : "Dicter un message"}
  aria-pressed={isRecording}
  onClick={toggleRecording}
>
  {isRecording ? <MicOff /> : <Mic />}
</button>
```

4.4 Bug #4 — Hard-Coded Language (P0)

This is the most consequential bug. `recognition.lang = 'fr-FR'` means the platform's entire Arabic-speaking audience — a significant portion of the Algerian market — gets gibberish when they try to dictate. The Copilot's three-mode system (pro/patient/enfant) does not change the recognition language. The fix is two-phase: Phase 1 adds a [FR/AR] manual toggle next to the mic button (2 hours); Phase 2 migrates to a server-side Whisper integration that can handle Darija (1 week). The Phase 1 fix is critical because the current state actively misleads Arabic-speaking users.

— CHAPTER 05 · VERDICT & FIX PLAN

Final Assessment & Phased Rollout

Voice Dictation is a **genuinely valuable feature for a clinical Copilot**, but the current implementation has four bugs that prevent it from being trustworthy across the platform's full audience. The happy path (Chrome user, French speaker, granted permission) works. Every other path silently fails. Total grade: **C+** — functional for the narrow happy path, broken for Firefox users, Arabic speakers, and reflexive-permission-deniers.

5.1 Phased Fix Plan

PHASE	FIXES	EFFORT	OUTCOME
Phase 1 · Now (Week 1)	Bugs #1, #2, #3 + [FR/AR] toggle (partial #4)	~6 hours	Grade lifts to B. Firefox users see a clean "non supporté". Arabic speakers can dictate. Permission denials reduced.
Phase 2 · Next (Quarter 2)	Bug #4 full fix: Whisper server-side for Darija	~1 week	Grade lifts to A-. Darija speakers get a working voice input. Privacy policy update needed.
Phase 3 · Later (Quarter 3)	Confidence-score highlighting + review toast	~3 days	Grade lifts to A. Clinical safety pattern: low-confidence words highlighted, mandatory review before send.

5.2 Cross-Feature Dependencies

- ✓ Feature #36 (AI Copilot Core) — voice input is the input layer for #36's chat interface; fixing #38 directly improves #36's UX

- ✓ Feature #37 (Copilot Mode System) — recognition language should follow mode (pro=FR, patient=AR, enfant=AR simplified)
- ✓ Feature #2 (State Management) — recording state should persist across refresh (currently resets to idle)
- ⚠ Feature #43 (Notifications Center) — could surface "microphone permission denied" as a persistent notification
- ⚠ Feature #60 (Accessibility Live Regions) — recording state should be announced via `aria-live="polite"` region

5.3 Final Assessment

1

MIC BUTTON PRESENT

3

LANGUAGES AUTO-DETECT (FR ONLY)

4

BUGS FOUND

C+

GRADE

~6h

PHASE 1 EFFORT

~1w

PHASE 2 EFFORT

FINAL VERDICT · C+ GRADE, 6 HOURS FROM B

Voice Dictation is **the right feature in the right place** — a clinical Copilot needs voice input, and the Algerian market needs it more than most. But the current implementation has four real bugs that prevent it from being trustworthy. The most consequential is the hard-coded `lang='fr-FR'` — Arabic speakers get gibberish, which is worse than no voice input at all (it actively misleads). Phase 1 fixes (6 hours total) lift the grade to B by adding Firefox detection, a permission pre-prompt, a stop-button aria-label fix, and a manual [FR/AR] language toggle. Phase 2 (Whisper server-side, 1 week) is the only path to Darija support. Phase 3 (confidence highlighting, 3 days) closes the clinical safety gap. The feature is worth investing in — voice is the future of clinical input.

Immediate Next Steps

1. Detect `!window.SpeechRecognition` on mount, render disabled state with tooltip (1 hour).
2. Add permission pre-prompt modal on first click (2 hours).
3. Fix stop button `aria-label` conditional (10 minutes).
4. Add [FR/AR] manual language toggle next to the mic button (2 hours).
5. Defer Darija support to Phase 2 (Whisper server-side, 1 week).

38

1 mic button, 4 bugs. Happy path works (Chrome + FR + granted permission). Arabic & Firefox broken. 6 hours of fixes lift to B. Whisper migration (1 week) lifts to A-.

5

CHAPTERS

3

SCREENSHOTS

4

BUGS FOUND

C+

GRADE
